



BITS Pilani
Pilani Campus

Machine Learning

AIML CZG565

M3 : Linear Models for Regression

Dr. Neha Vinayak

Course Faculty of MTech Cluster

BITS – CSIS - WILP

Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- We here by acknowledge all the contributors for their material and inputs.
- We have provided source information wherever necessary
- Students are requested to refer to the textbook w.r.t detailed content of the presentation deck shared over canvas
- We have reduced the slides from canvas and modified the content flow to suit the requirements of the course and for ease of class presentation

Source: “Probabilistic Machine Learning, An Introduction”, Kevin P. Murphy, Slides of Prof.Sugata, Prof. Chetana from BITS Pilani, Prof. Raja vadhana from BITS Pilani , CS109 and CS229 stanford lecture notes and many others who made their course materials freely available online.

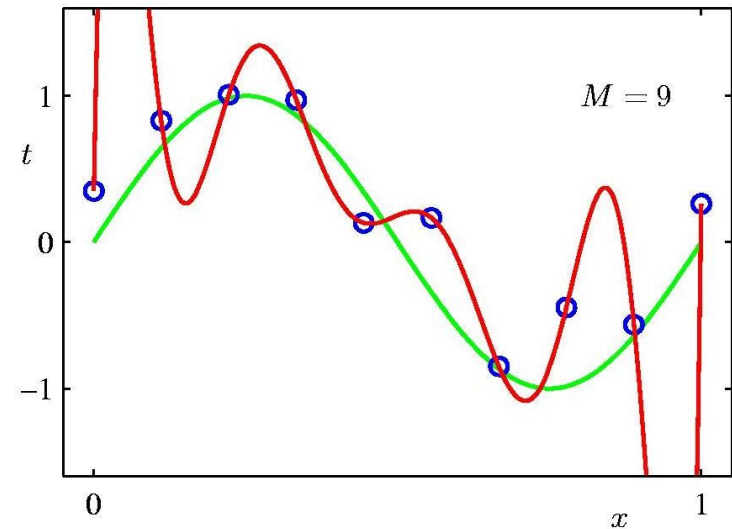
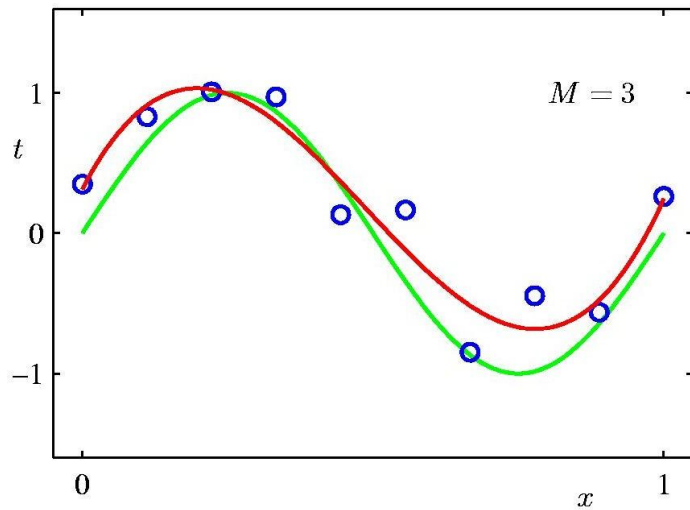
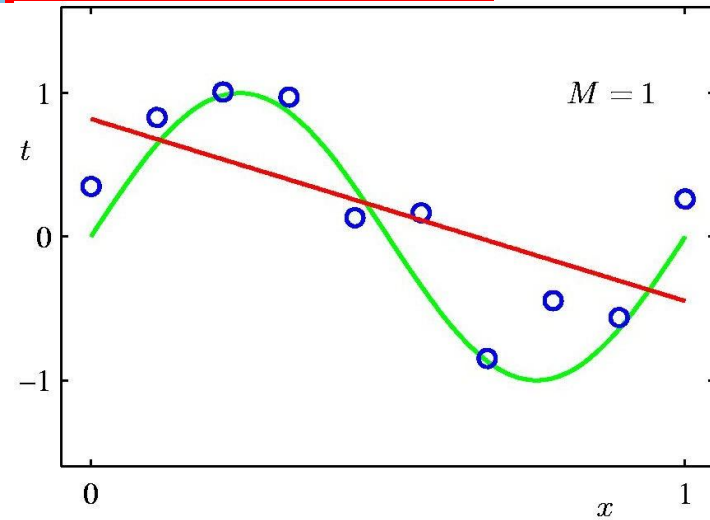
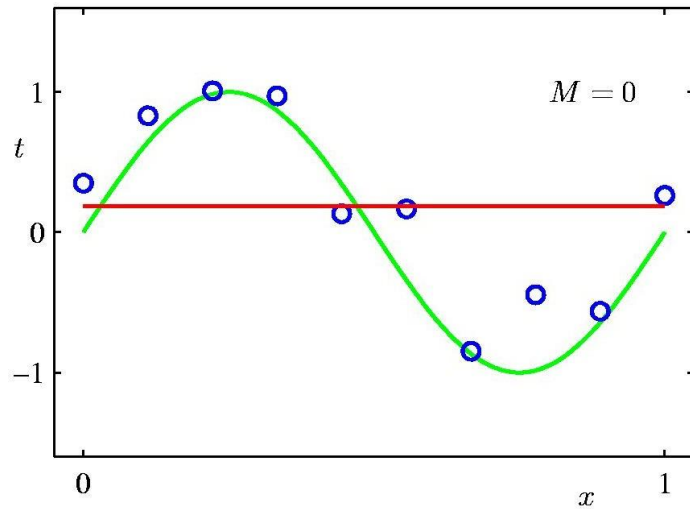
Course Plan

M1	Introduction
M2	Machine learning Workflow
M3	Linear Models for Regression
M4	Linear Models for Classification
M5	Decision Tree
M6	Instance Based Learning
M7	Support Vector Machine
M8	Bayesian Learning
M9	Ensemble Learning
M10	Unsupervised Learning
M11	Machine Learning Model Evaluation/Comparison

Linear Basis Models

What if output is a non-linear function of input vector?

Polynomial Regression



Linear Basis Function Models



- The inputs **X** for linear regression can be:
 - Original quantitative inputs
 - Transformation of quantitative inputs
 - e.g. log, exp, square root, square, etc.
 - Polynomial transformation
 - example: $y = \beta_0 + \beta_1 \cdot x + \beta_2 \cdot x^2 + \beta_3 \cdot x^3$
 - Basis expansions
 - Dummy coding of categorical inputs
 - Interactions between variables
 - example: $x_3 = x_1 \cdot x_2$

This allows use of linear regression techniques to fit non-linear datasets.

Input X	Output Y
exp(2)	
exp(4)	
exp(6.3)	
exp(9.2)	

X No.of.Years of Experience (in Years)	X^2	Y Salary Of the Employee (in Lakhs)
1	1	2
2	4	3
3	9	4
4	16	5
5	25	6

X1 = Graduate	X2 = PostGraduate	X3 = Others	Y Salary Of the Employee
0	0	1	2
1	0	0	3
0	0	1	4
0	1	0	5
1	0	0	6

Linear Basis Function Models



Example: an M-th order polynomial function of one dimensional feature x :

$$y(x, \mathbf{w}) = \underbrace{w_0} + \sum_{j=1}^M \underbrace{w_j x^j}_{\text{Lin Rep.}}$$

where x^j = j-th power of x

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \dots + w_M x^M = \sum_{j=0}^M w_j x^j$$

X No.of.Years of Experience (in Years)	X^2	Y Salary Of the Employee (in Lakhs)
1	1	2
2	4	3
3	9	4
4	16	5
5	25	6

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \underline{\underline{\phi(\mathbf{x})}}$$

$\phi_j(x)$ are known as *basis functions*. Typically, $\phi_0(x) = 1$, so that w_0 acts as a bias.

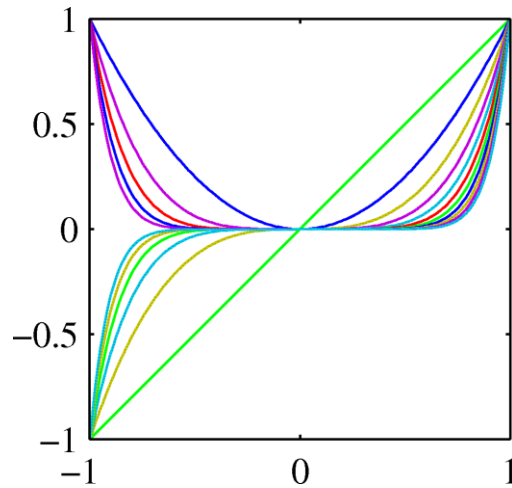
$$h_0(x) = \sum_j \phi_j(x)$$

In the simplest case, we use linear basis functions : $\phi_d(x) = x_d$.

They are called linear models because this function is

linear in $\mathbf{w}$.

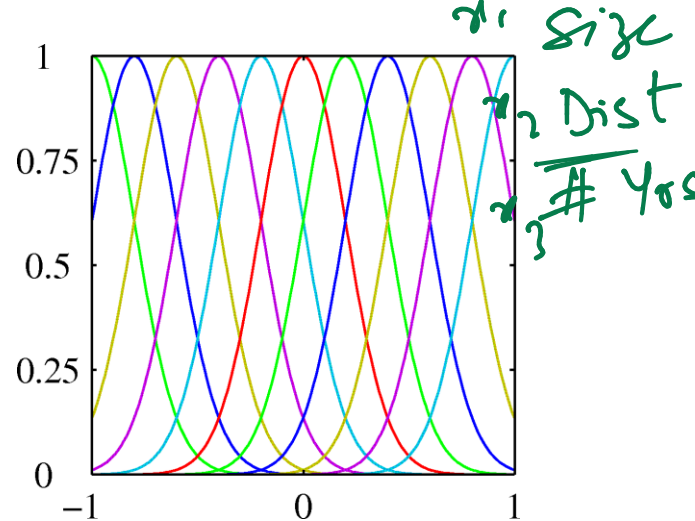
Linear Basis Function Models - Examples



Polynomial basis functions:

$$\phi_j(x) = x^j.$$

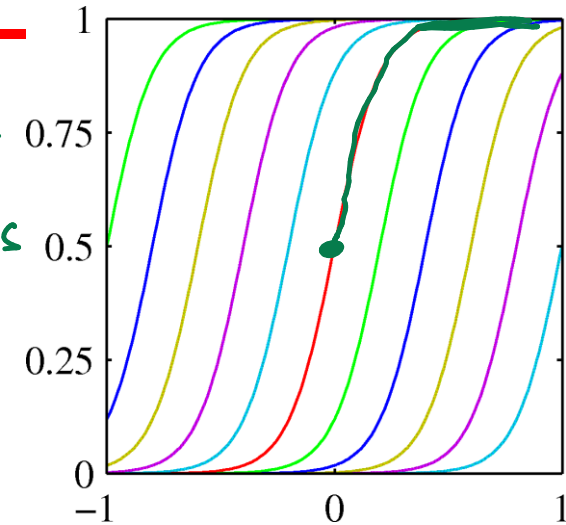
These are global; a small change in x affect all basis functions.



Gaussian basis functions:

$$\phi_j(x) = \exp \left\{ -\frac{(x - \mu_j)^2}{2s^2} \right\}$$

These are local; a small change in x only affect nearby basis functions. μ_j and s control location and scale (width).



Sigmoidal basis functions:

where $\phi_j(x) = \sigma \left(\frac{x - \mu_j}{s} \right)$

$$\sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Also these are local; a small change in x only affect nearby basis functions. μ_j and s control location and scale (slope).

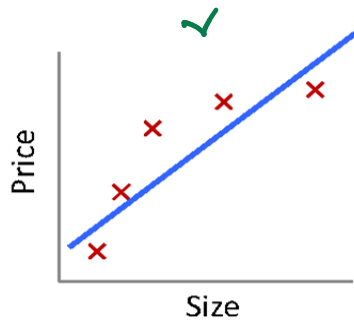
Notion of Bias - Variance

Quality of Fit

$$\text{bias} = \text{err}$$

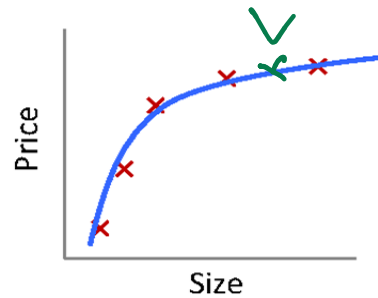
$$\text{var} = \text{tr} + \text{test}$$

train $\rightarrow$ good
test $\rightarrow$ bad



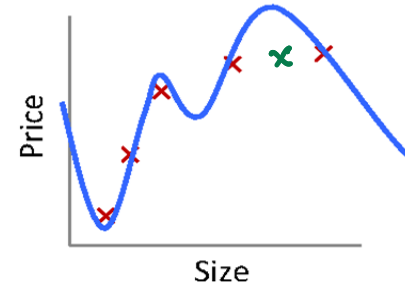
$$\theta_0 + \theta_1 x$$

Underfitting
(high bias)



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

Correct fit



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

Overfitting

(high variance)

Complex

generalizing to test

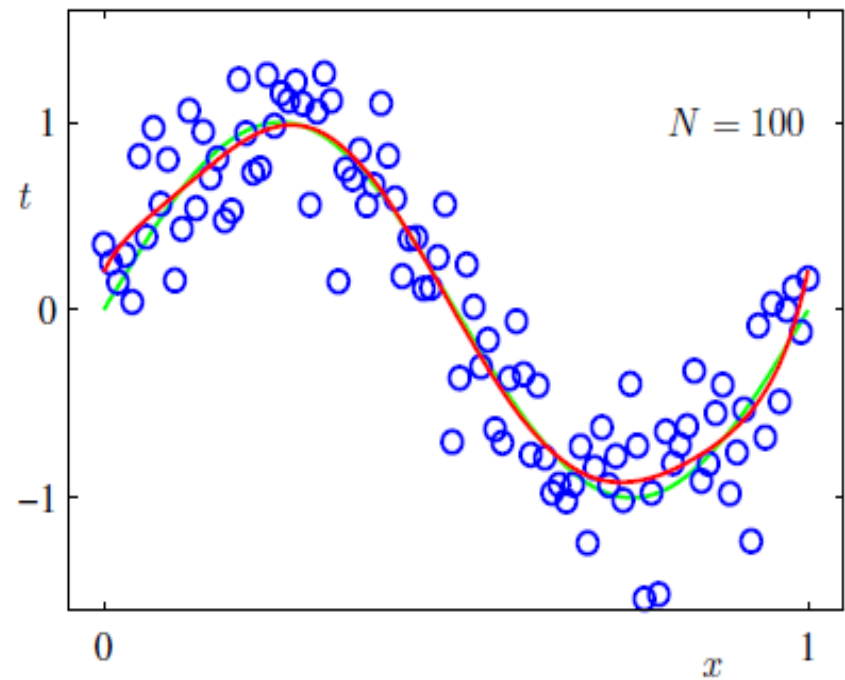
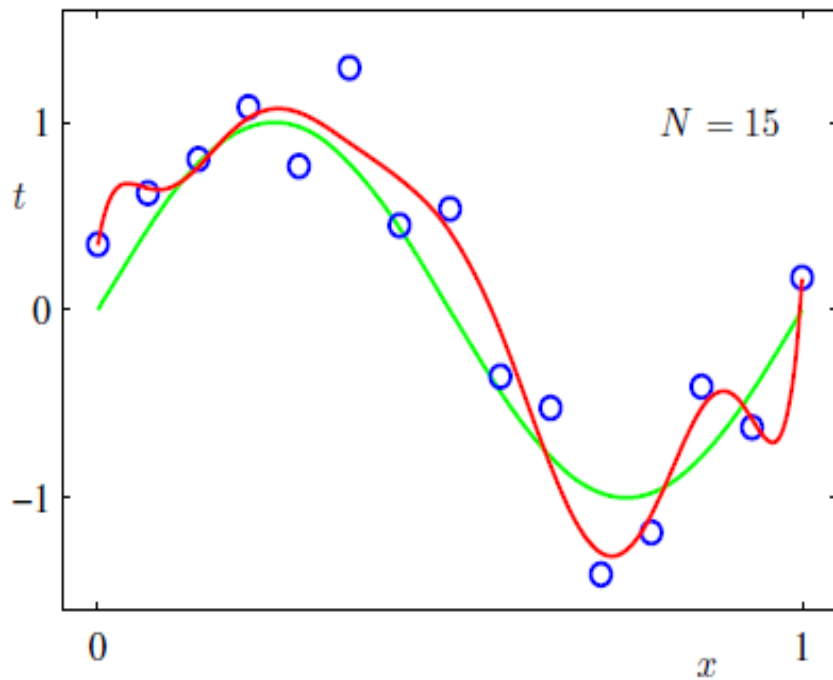
Overfitting:

- The learned hypothesis may fit the training set very well ($J(\theta) \approx 0$)
- ...but fails to generalize to new examples

bias-var tradeoff

Handling Overfitting – Way 1

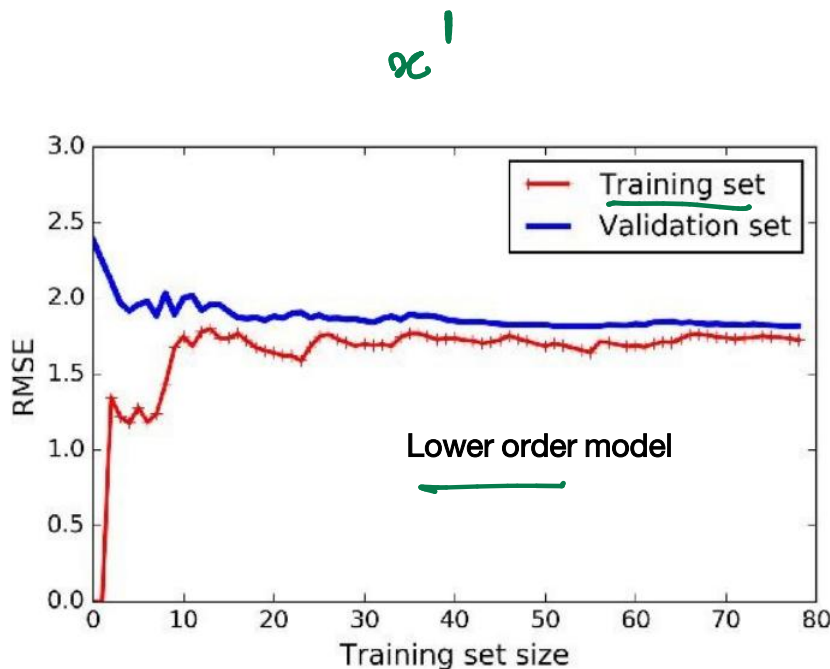
Increase in Size of the data set reduces the over-fitting problem



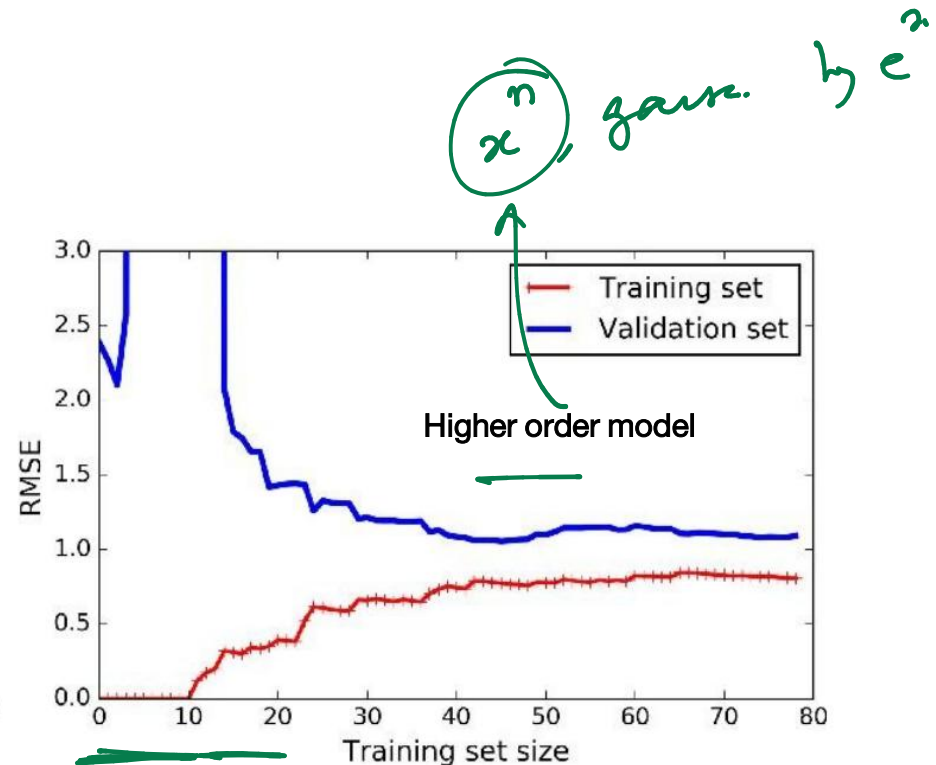
Effect of Training Size on Over fitting

Problem Type 4 : Interpretation of the Model Fit

- Size of training dataset needs to be large to prevent overfitting when higher order model is used.



Better fit



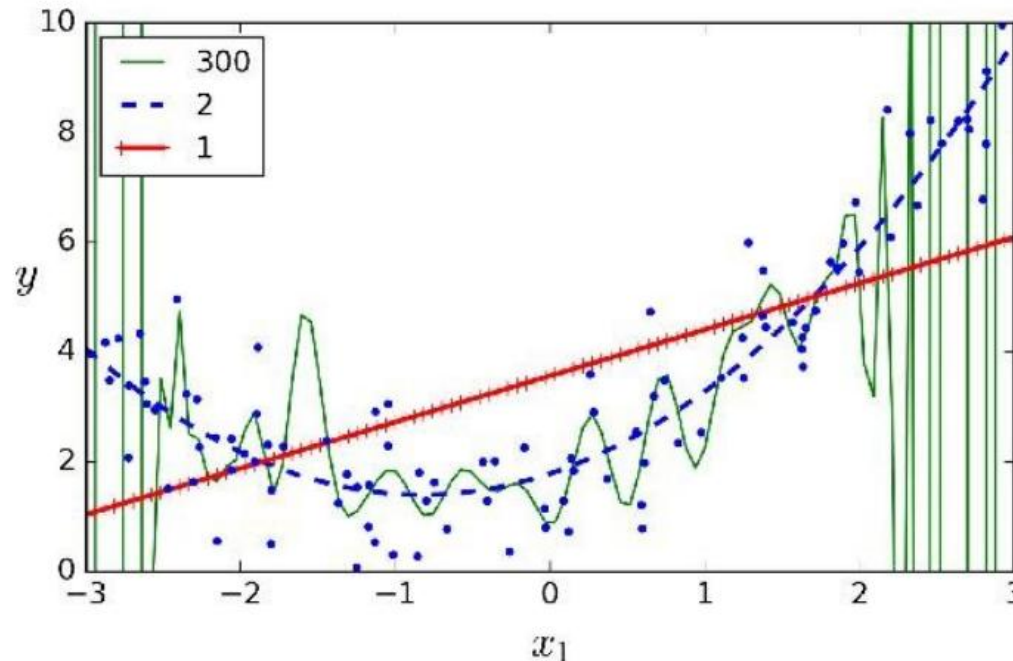
OV

Polynomial Fitting can lead to Overfitting

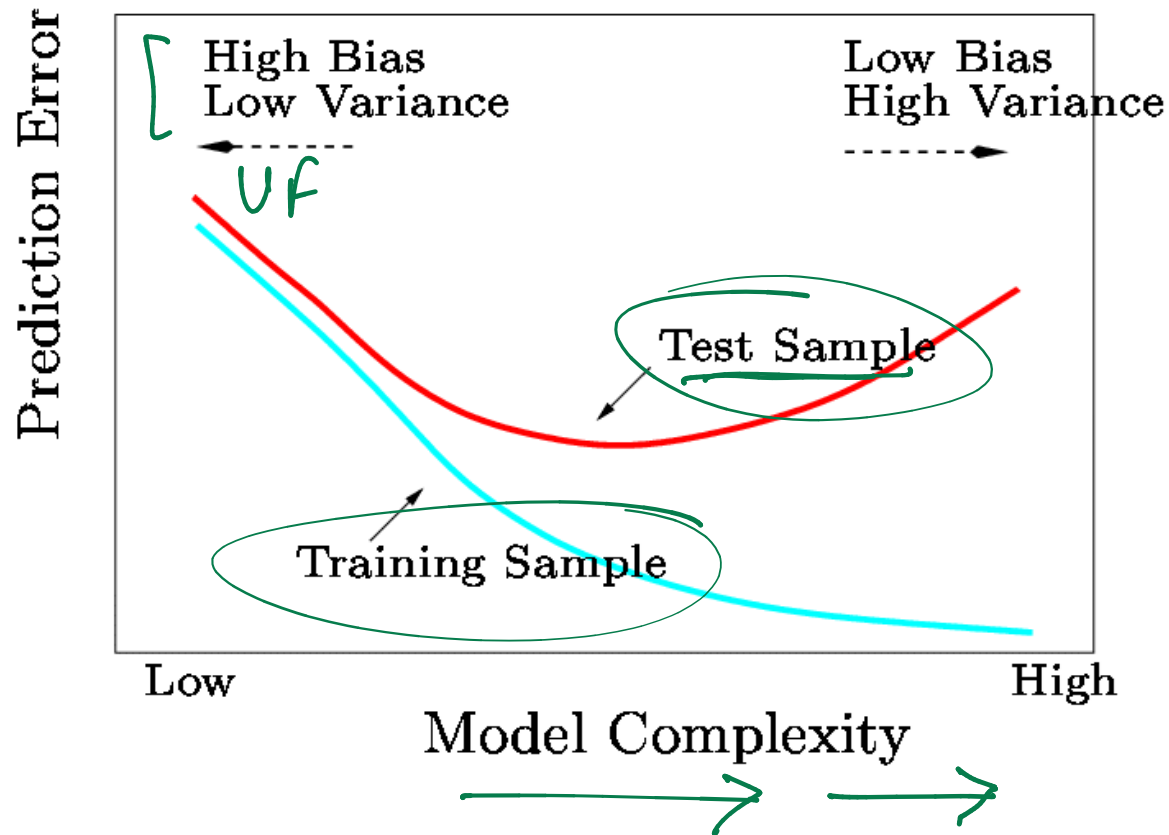


Handling Overfitting – Way 2 – Reduce the complexity of the model

- Underlying target function is quadratic
- Linear model results in under fitting with large bias
- Polynomial of order 300 results in a large variance



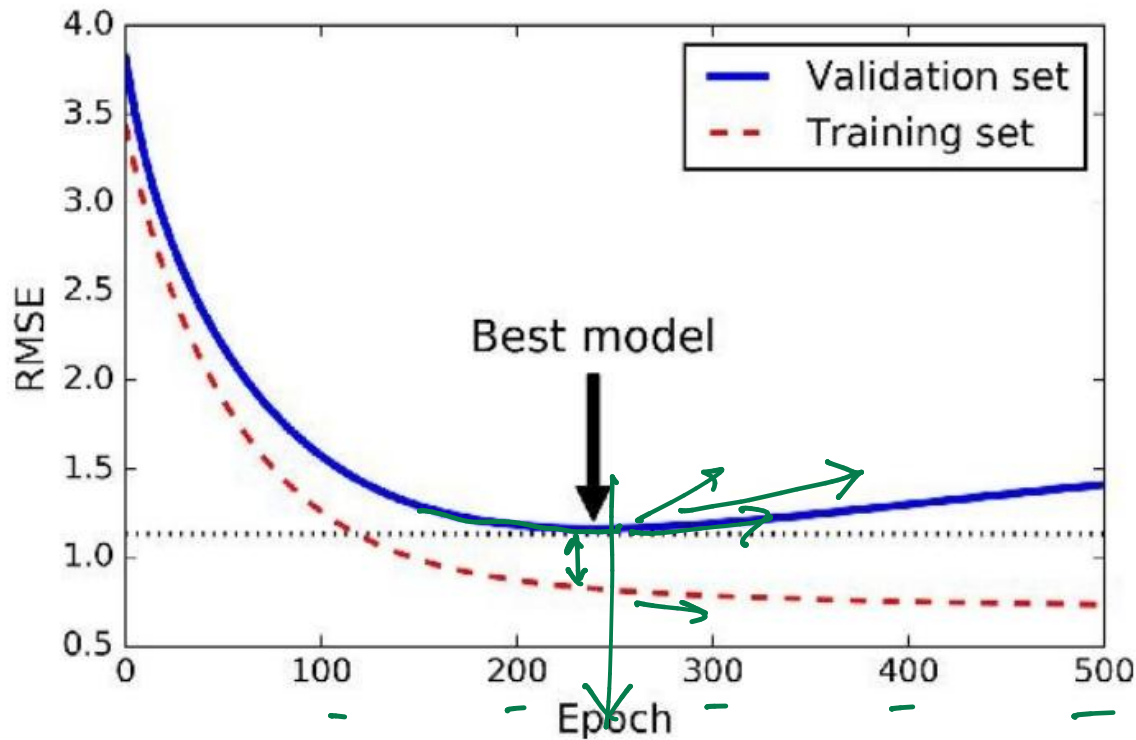
How to experiment on Model Complexity ?



Handling Overfitting – Way 3

Early Stop the Training

epochs.



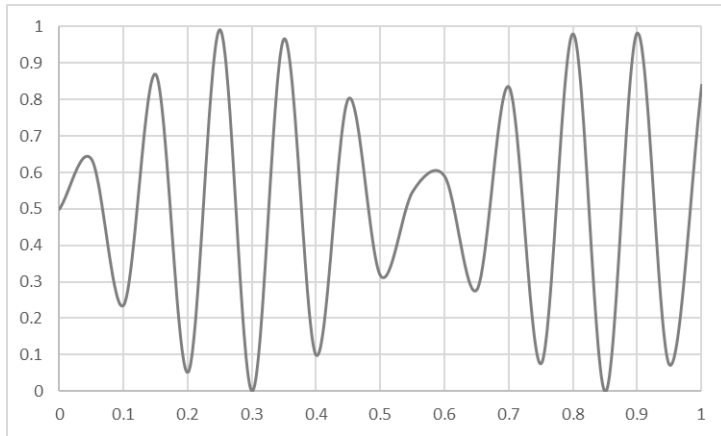
Regularization

(*This notion is common for both Linear Regression – Module 3
and Logistic Regression – Module 4)

Overfitting vs Underfitting

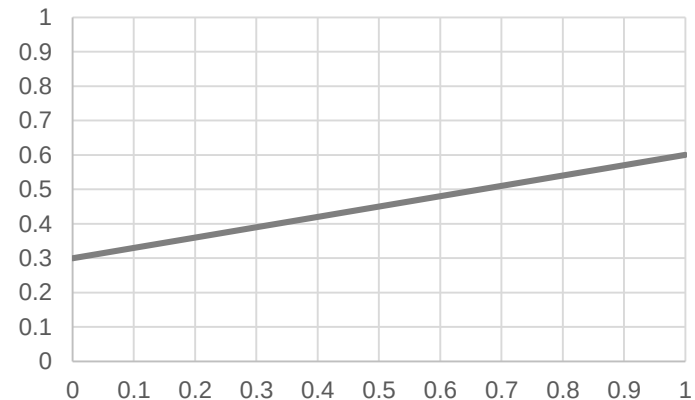
Overfitting

- Fitting the data too well
 - Features are noisy / uncorrelated to concept



Underfitting

- Learning too little of the true concept
 - Features don't capture concept
 - Too much bias in model



Regularization



- A method for automatically controlling the complexity of the learned hypothesis
- **Idea:** penalize for large values of θ_j
 - Can incorporate into the cost function
 - Works well when we have a lot of features, each that contributes a bit to predicting the label
- Can also address overfitting by eliminating features (either manually or via model selection)

Regularization



- Linear regression objective function

$$\min_{\theta} \quad \underbrace{J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}_{\text{model fit to data}} + \underbrace{\frac{\lambda}{2} \sum_{j=1}^d \theta_j^2}_{\text{regularization}} \quad \text{penalty}$$

- λ is the regularization parameter ($\lambda \geq 0$)
- No regularization on θ_0 !

$\lambda \rightarrow \text{very small} \rightarrow \text{little effect}$
 $\lambda \rightarrow \text{v. large} \rightarrow \text{drives } \theta \rightarrow 0$

Understanding Regularization



$$J(\theta) = \left[\frac{1}{2n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2 \right] \rightarrow 9 + 15$$

Handwritten notes: "min" above the first term, "10" above the first term, "20" above the second term, "wet. small" to the right of the second term.

- Note that $\sum_{j=1}^d \theta_j^2 = \|\theta_{1:d}\|_2^2$
– This is the magnitude of the feature coefficient vector!

$$\begin{bmatrix} \theta^2 \end{bmatrix}$$

Handwritten notes: "sm LA" below the equation.

- We can also think of this as:

$$\sum_{j=1}^d (\theta_j - 0)^2 = \|\theta_{1:d} - \vec{0}\|_2^2$$

Handwritten notes: "min" below the equation, $\begin{bmatrix} \theta - \vec{0} \end{bmatrix}$ to the right, and "SAR" below that.

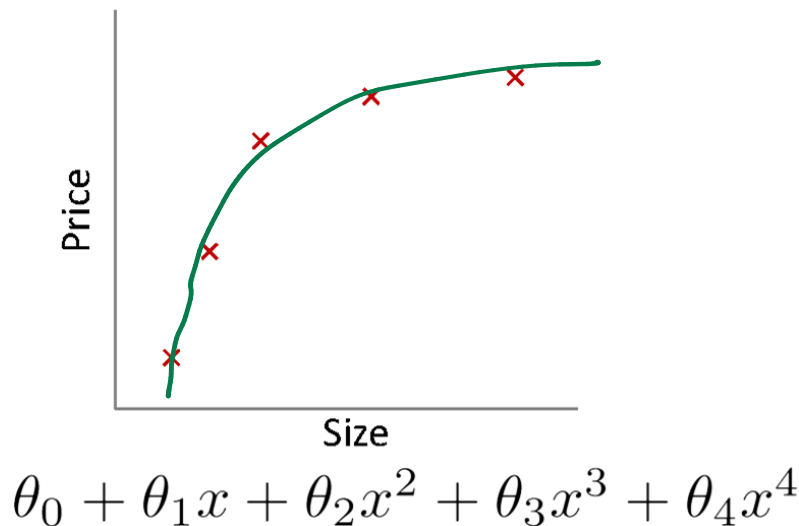
- L_2 regularization pulls coefficients toward 0

Understanding Regularization



$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right)^2 + \underbrace{\frac{\lambda}{2} \sum_{j=1}^d \theta_j^2}_{\text{v. large}}$$

- What happens if we set λ to be huge (e.g., 10^{10})?



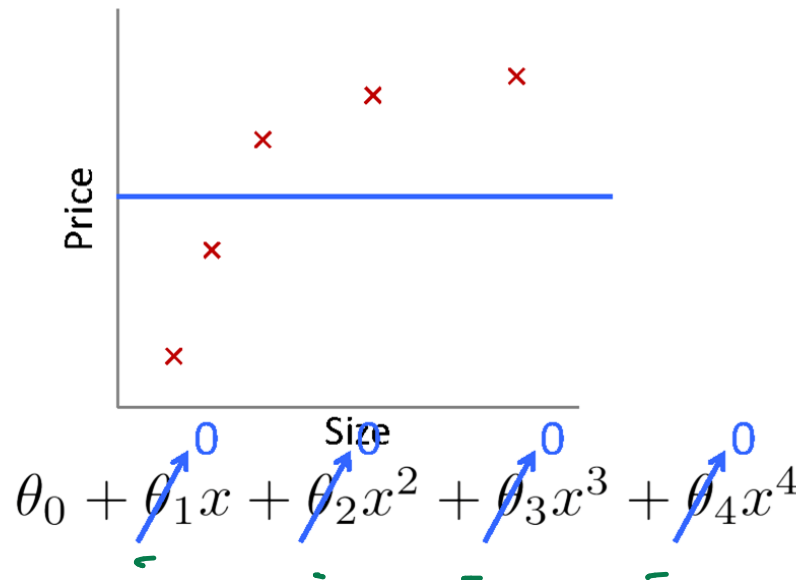
$[\theta - \vec{0}]$

Understanding Regularization



$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2$$

- What happens if we set λ to be huge (e.g., 10^{10})?



Based on example by Andrew Ng

Regularization

Ridge Regression / Tikhonov regularization



- Cost Function

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2$$

L2 Reg.

↳ only $\theta_1 \dots \theta_d$
not on θ_0

- Fit by solving $\min J(\theta)$

- Gradient update:

$$\frac{\partial}{\partial \theta_0} J(\theta) \quad \theta_0 \leftarrow \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \quad \checkmark$$

$$\frac{\partial}{\partial \theta_j} J(\theta) \quad \theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)} - \alpha \lambda \theta_j$$

Rewritten:

$$\theta_j \leftarrow \underbrace{\theta_j(1 - \alpha \lambda)} - \alpha \cdot \frac{1}{n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

Regularization

Lasso Regression (Least Absolute Shrinkage and Selection Operator Regression)

- Cost Function

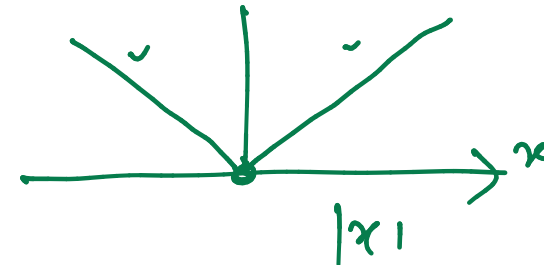
$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \lambda \sum_j |\theta_j|$$

- Fit by solving $\min J(\theta)$

- Gradient update:

$$\frac{\partial}{\partial \theta_0} J(\theta) \quad \theta_0 \leftarrow \theta_0 - \alpha \cdot \frac{1}{n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)$$

$$\frac{\partial}{\partial \theta_j} J(\theta) \quad \theta_j \leftarrow \theta_j - \alpha \cdot \frac{1}{n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)} - \alpha \lambda \cdot \text{sign}(\theta_j)$$



✓ Feature Selector.

$$\text{sign}(\theta_j) = \begin{cases} -1, & \theta_j < 0 \\ 0, & \theta_j = 0 \\ +1, & \theta_j > 0 \end{cases}$$

Regularization

Elastic Net



- Cost Function

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \underbrace{r\lambda \sum_j |\theta_j|}_{\mathcal{L}} + \underbrace{\frac{(1-r)\lambda}{2} \sum_j \theta_j^2}_{\mathcal{R}}$$

Control the regularization using the **Mix Ratio “r”**:

$r = 0 \rightarrow$ equivalent to **Ridge Regression**;

$r = 1 \rightarrow$ equivalent to **Lasso Regression**

- Gradient update:

$$\frac{\partial}{\partial \theta_0} J(\theta) \quad \theta_0 \leftarrow \theta_0 - \alpha \cdot \frac{1}{n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)$$

$$\frac{\partial}{\partial \theta_j} J(\theta) \quad \theta_j \leftarrow \theta_j - \alpha \cdot \frac{1}{n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)} - \alpha r \lambda \cdot \text{sign}(\theta_j) - \alpha (1-r) \lambda \theta_j$$

`from sklearn.linear_model import ElasticNet
ElasticNet(alpha=0.1, l1_ratio=0.5)`

How to choose the right Regularization?

Common Usage & Observation



- L1 regularization has the ability to set some coefficients to 0 exactly leading to a *sparse* model
- L1 regularization helps in feature selection by eliminating the features that are not important
- **L1 cannot be used efficiently in gradient-based approaches since it is not-differentiable unlike L2**
- L2 will in general lead to small magnitudes of weights but not exactly 0.
- Elastic net – Prefer in Highly correlated features



Numerical Exercise – For Student Practice

Fit a linear regression. Show only the first iteration of Gradient descent algorithm using learning rate of **0.02** for the following data, if the Relative Risk of Coronary Heart Disease is believed to be only linearly dependent on BMI as well as Diastolic Pressure. Assume the intercept of the regression model as **5** and the slope of independent variables as **-0.03 (negative)**.

Patient	Systolic Pressure mm Hg	Diastolic Pressure mm Hg	BMI	Waist Circumference Threshold cm	RR-CHD (Relative Risk of Coronary Heart Disease)
1	140	80	35	100	1.81
2	120	80	25	80	1.22
3	130	100	30	60	1.71

Apply a regularization on the same problem with regularization constant 5 and apply GD for 2 iterations & interpret the results. Try both ridge regression as well as lasso regression. Below equation is changed only for ridge regression. Students must apply appropriate GD update equation changes for this problem.

Steps :

1. Identification of the equations $y = w_0 + w_1X_1 + w_2X_2$
2. Cost function & derivative
 1. $W_0' = w_0 - 1/3 * \text{learning rate} * (\text{sum}(w_0 + w_1X_1 + w_2X_2 - y))$
 2. $W_1' = w_1 * (1 - \text{learning rate} * \text{regularization constant}) - 1/3 * \text{learning rate} * (\text{sum}(w_0 + w_1X_1 + w_2X_2 - y) * x_1)$
 3. $W_2' = w_2 * (1 - \text{learning rate} * \text{regularization constant}) - 1/3 * \text{learning rate} * (\text{sum}(w_0 + w_1X_1 + w_2X_2 - y) * x_2)$
3. Apply the equations

$$J(\theta) = \frac{1}{2n} \sum (\hat{y} - y)^2 + \frac{\lambda}{2} \sum \theta^2 \quad \downarrow$$

$\downarrow$
 min err.

$\theta \uparrow \downarrow$

min

$\text{Cor.} - N. \rightarrow \text{Hospital} \rightarrow \text{noise}$

$\hookrightarrow \text{repo} + \text{noise} = \text{Gen?}$



Python Lab Exercise – For Student Practice

Labware

Go to your virtual lab file under : Machine Learning ☐ LabCapsule 2 Linear Polynomial Regression ☐ 3 Polynomial Regression

Download ML_Lab 5 PolynomialRegression.ipynb

Try to change the degree to {2, 5, 7, 10} in the below function line :
PolynomialFeatures(degree=3)

Observe and interpret on the performance of the training data

Note: Training vs Test Data split is not coded in this implementation. Below are the suggestions to experiment further:

- Add few hundreds of data (Use synthetic data generation python libraries available : Refer here)
- Split the data into 80% train vs 20% test set
- Built the polynomial model using above four different degree on training set
- For each of the model apply in the test set
- Find the MSE for both training data and separately for test data
- Plot the four experiment results in a plot and interpret the notion of overfit vs underfit
- Suggest which degree is a best fit .

References

- T1 - Chapter 1 – Machine Learning, Tom Mitchell
- Chapter 1, 2 – Introduction to Machine Learning, 2nd edition, Ethem Alpaydin
- R1 – Chapter #1, # 3,#4 (Christopher M. Bhisop, Pattern Recognition & Machine Learning) & Refresh your MFDS course basics

Thank you !